

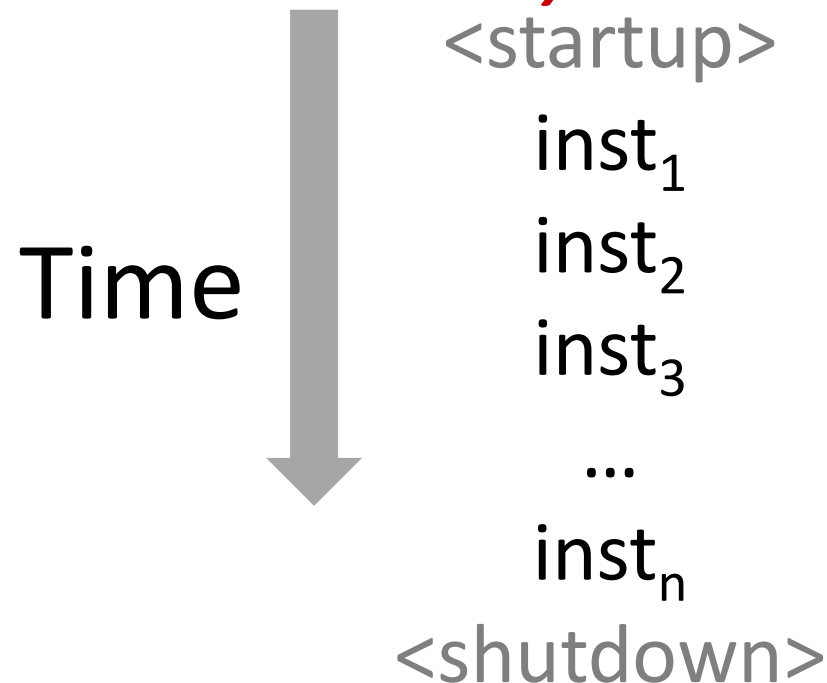
# Lecture 8. Exceptions, Processes, Threads

# Control Flow

- **Processors do only one thing:**

- From startup to shutdown, a CPU simply reads and executes (interprets) a sequence of instructions, one at a time
- This sequence is the CPU's *control flow* (or *flow of control*)

*Physical control flow*



# Altering the Control Flow

- **Up to now: two mechanisms for changing control flow:**

- Jumps and branches
- Call and return

Both react to changes in *program state*

- **Insufficient for a useful system:**

**Difficult to react to changes in *system state***

- data arrives from a disk or a network adapter
- instruction divides by zero
- user hits Ctrl-C or Ctrl-Alt-Del at the keyboard
- System timer expires

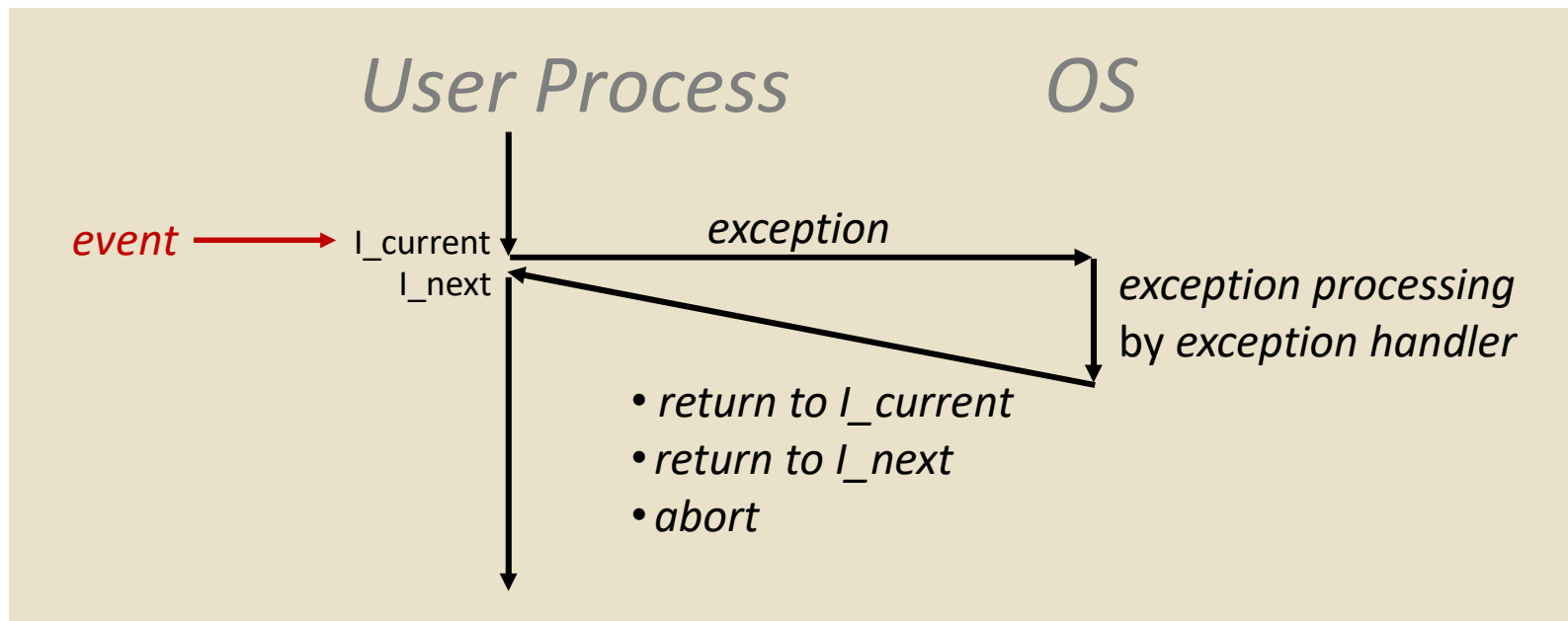
- **System needs mechanisms for “exceptional control flow”**

# Exceptional Control Flow

- **Exists at all levels of a computer system**
- **Low level mechanisms**
  - Exceptions
    - change in control flow in response to a system event (i.e., change in system state)
  - Combination of hardware and OS software
- **Higher level mechanisms**
  - Process context switch
  - Signals
  - Nonlocal jumps: `setjmp()/longjmp()`
  - Implemented by either:
    - OS software (context switch and signals)
    - C language runtime library (nonlocal jumps)

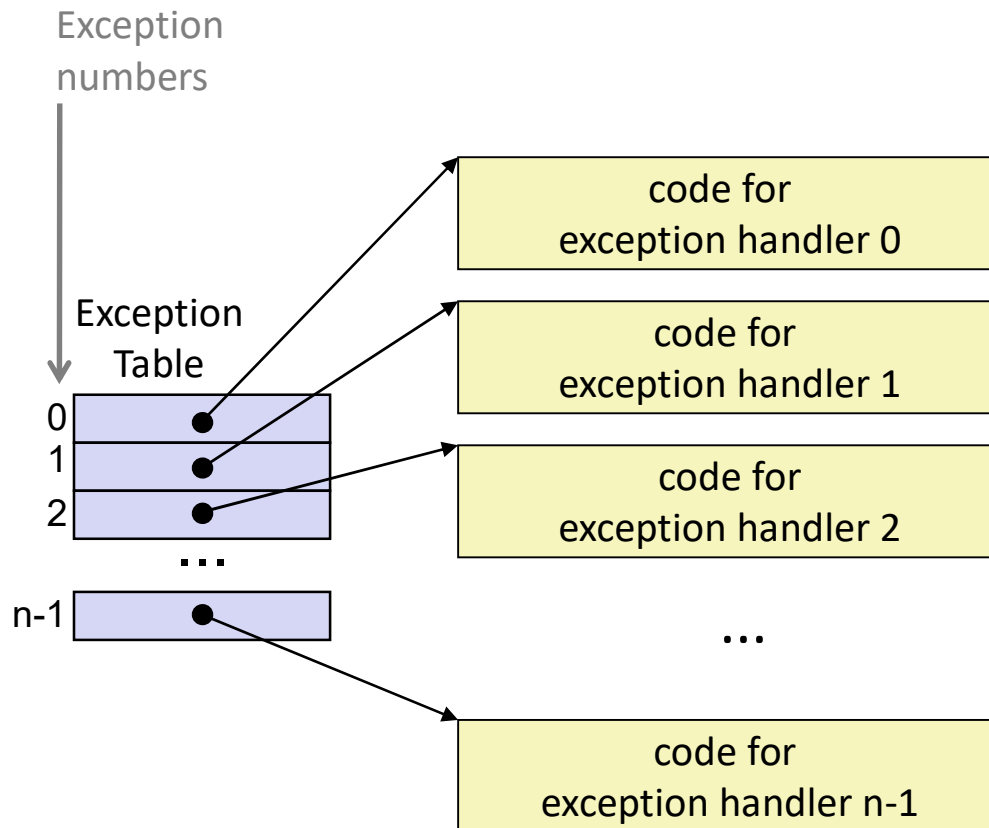
# Exceptions

- An **exception** is a transfer of control to the OS in response to some *event* (i.e., change in processor state)



- **Examples:**  
div by 0, arithmetic overflow, page fault, I/O request completes, Ctrl-C

# Exception Tables



- Each type of event has a unique exception number  $k$
- $k$  = index into exception table (a.k.a. interrupt vector)
- Handler  $k$  is called each time exception  $k$  occurs

# Asynchronous Exceptions (Interrupts)

- **Caused by events external to the processor**

- Indicated by setting the processor's interrupt pin
- Handler returns to "next" instruction

- **Examples:**

- I/O interrupts
  - hitting Ctrl-C at the keyboard
  - arrival of a packet from a network
  - arrival of data from a disk
- Hard reset interrupt
  - hitting the reset button
- Soft reset interrupt
  - hitting Ctrl-Alt-Delete on a PC

# Synchronous Exceptions

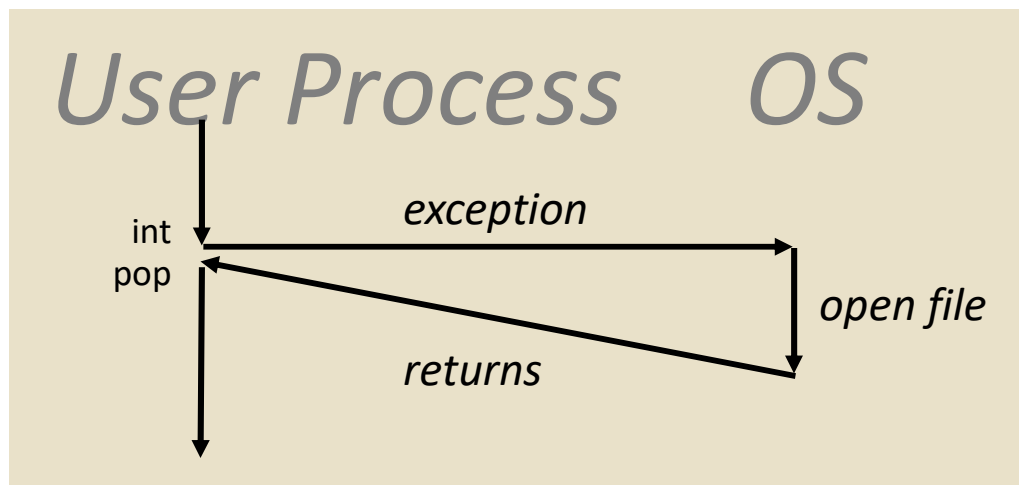
- Caused by events that occur as a result of executing an instruction:
  - **Traps**
    - Intentional
    - Examples: **system calls**, breakpoint traps, special instructions
    - Returns control to “next” instruction
  - **Faults**
    - Unintentional but possibly recoverable
    - Examples: page faults (recoverable), protection faults (unrecoverable), floating point exceptions
    - Either re-executes faulting (“current”) instruction or aborts
  - **Aborts**
    - unintentional and unrecoverable
    - Examples: parity error, machine check
    - Aborts current program



# Trap Example: Opening File

- User calls: `open(filename, options)`
- Function `open` executes system call instruction `int`

```
0804d070 <__libc_open>:  
. . .  
804d082:      cd 80          int      $0x80  
804d084:      5b              pop      %ebx  
. . .
```



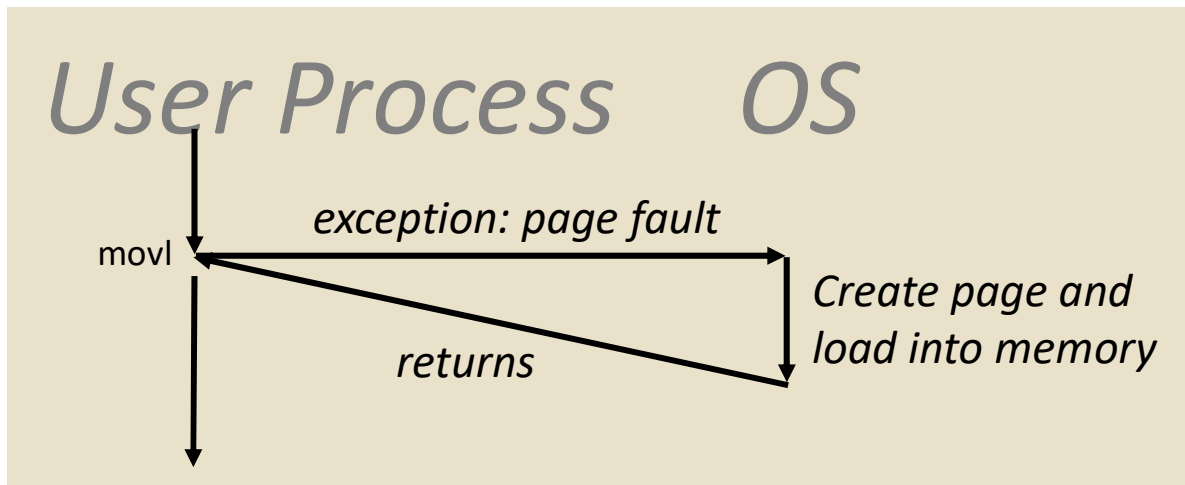
- OS must find or create file, get it ready for reading or writing
- Returns integer file descriptor

# Fault Example: Page Fault

- User writes to memory location
- That portion (page) of user's memory is currently on disk

```
int a[1000];  
main ()  
{  
    a[500] = 13;  
}
```

```
80483b7:      c7 05 10 9d 04 08 0d  movl    $0xd,0x8049d10
```

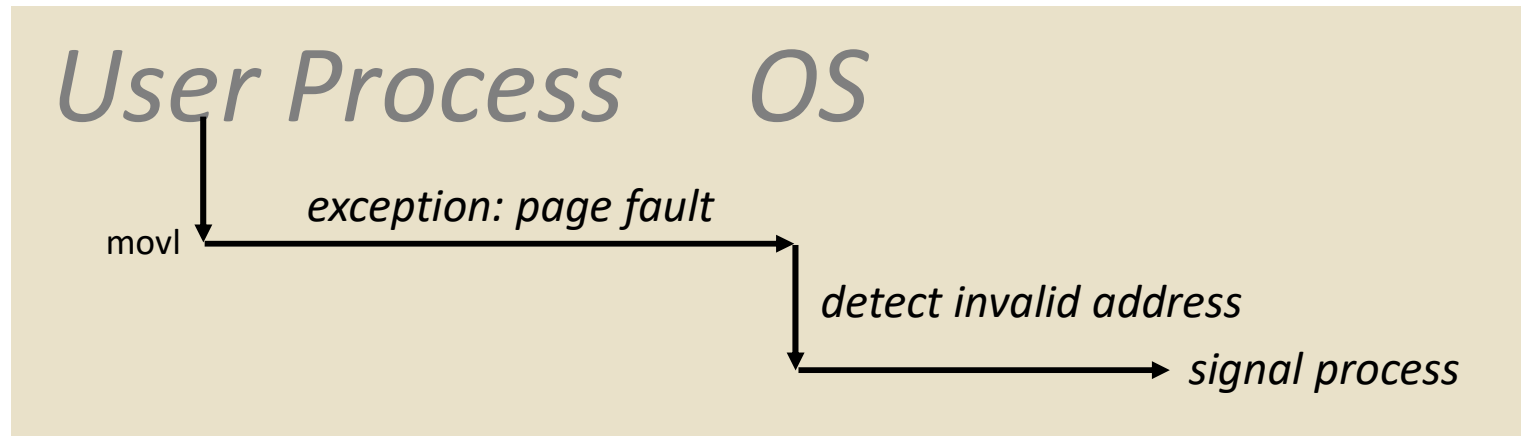


- Page handler must load page into physical memory
- Returns to faulting instruction
- Successful on second try

# Fault Example: Invalid Memory Reference

```
int a[1000];  
main ()  
{  
    a[5000] = 13;  
}
```

80483b7: c7 05 60 e3 04 08 0d movl \$0xd,0x804e360



- Page handler detects invalid address
- Sends **SIGSEGV** signal to user process
- User process exits with “segmentation fault”

# Exception Table IA32 (Excerpt)

| <i>Exception Number</i> | <i>Description</i>       | <i>Exception Class</i> |
|-------------------------|--------------------------|------------------------|
| 0                       | Divide error             | Fault                  |
| 13                      | General protection fault | Fault                  |
| 14                      | Page fault               | Fault                  |
| 18                      | Machine check            | Abort                  |
| 32-127                  | OS-defined               | Interrupt or trap      |
| 128 (0x80)              | System call              | Trap                   |
| 33 (0x21)               | (Windows)                | Trap                   |
| 129-255                 | OS-defined               | Interrupt or trap      |

Check Table 6-1:

<http://download.intel.com/design/processor/manuals/253665.pdf>

# Processes

- **Definition: A *process* is an instance of a running program.**
  - One of the most profound ideas in computer science
  - Not the same as “program” or “processor”
- **Process provides each program with two key abstractions:**
  - Logical control flow
    - Each program seems to have exclusive use of the CPU
  - Private virtual address space
    - Each program seems to have exclusive use of main memory
- **How are these Illusions maintained?**
  - Process executions interleaved (multitasking) or run on separate cores
  - Address spaces managed by virtual memory system

# INT 21h – The Gateway to MS-DOS

- In the 16-bit era (MS-DOS), INT 21h was the **primary Application Programming Interface (API)**. It provided all "DOS Services," including file management, console I/O, and program execution control. It acted as a "software gate." Instead of calling a memory address (which might change), the programmer just called INT 21h, and the OS handled the rest. While modern 64-bit **Windows** no longer uses INT 21h, it remains the architectural "ancestor" of how software communicates with a kernel.
- **Parameters for INT 21h** were not passed on a stack (like in C) but directly in CPU Registers. The value in the AH register determined which service was being requested.

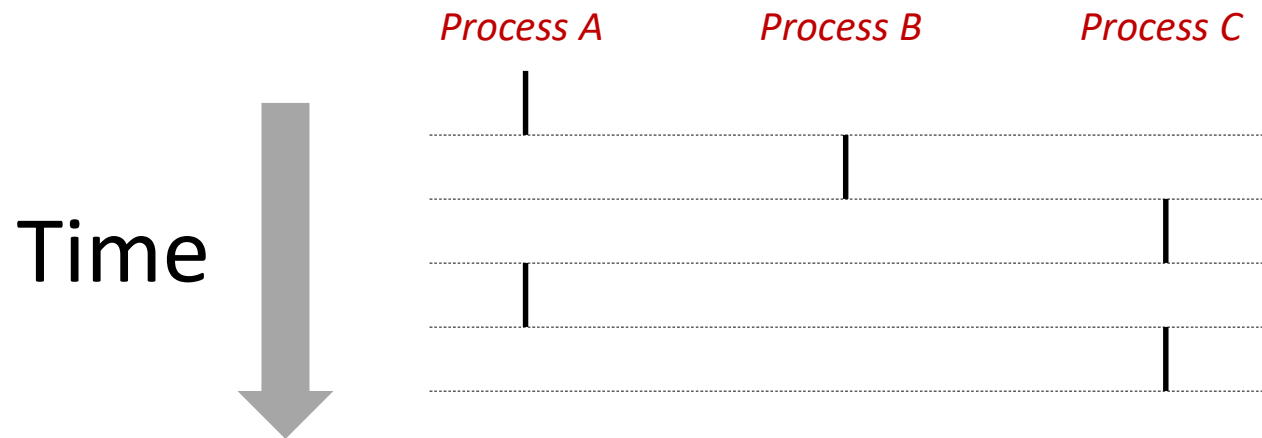
```
mov ah, 09h      ; Select "Print String" service
mov dx, offset msg ; Provide the address of the data
int 21h          ; Trigger the software interrupt
```

# Modern "Interrupts" – SYSCALL and SYSENTER

- In modern **Windows**, the transition from User Mode to Kernel Mode is handled by specialized hardware instructions: **SYSCALL** (AMD/Intel 64-bit) or **SYSENTER** (Intel 32-bit).
- **The Process:**
  1. Application calls a Win32 function.
  2. The function calls a "Native API" in ntdll.dll.
  3. ntdll.dll puts the system call number in the RAX register and executes SYSCALL.
  4. The CPU immediately jumps to a fixed location in the Kernel.

# Concurrent Processes

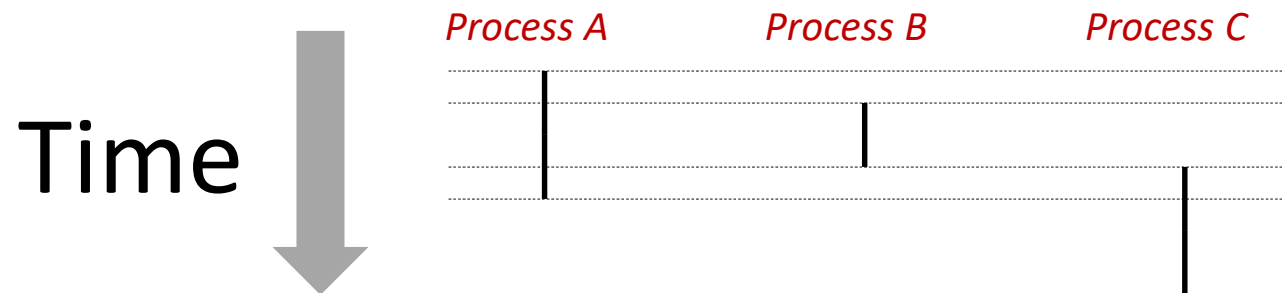
- Two processes *run concurrently* (are concurrent) if their flows overlap in time
- Otherwise, they are *sequential*
- Examples (running on single core):
  - Concurrent: A & B, A & C
  - Sequential: B & C





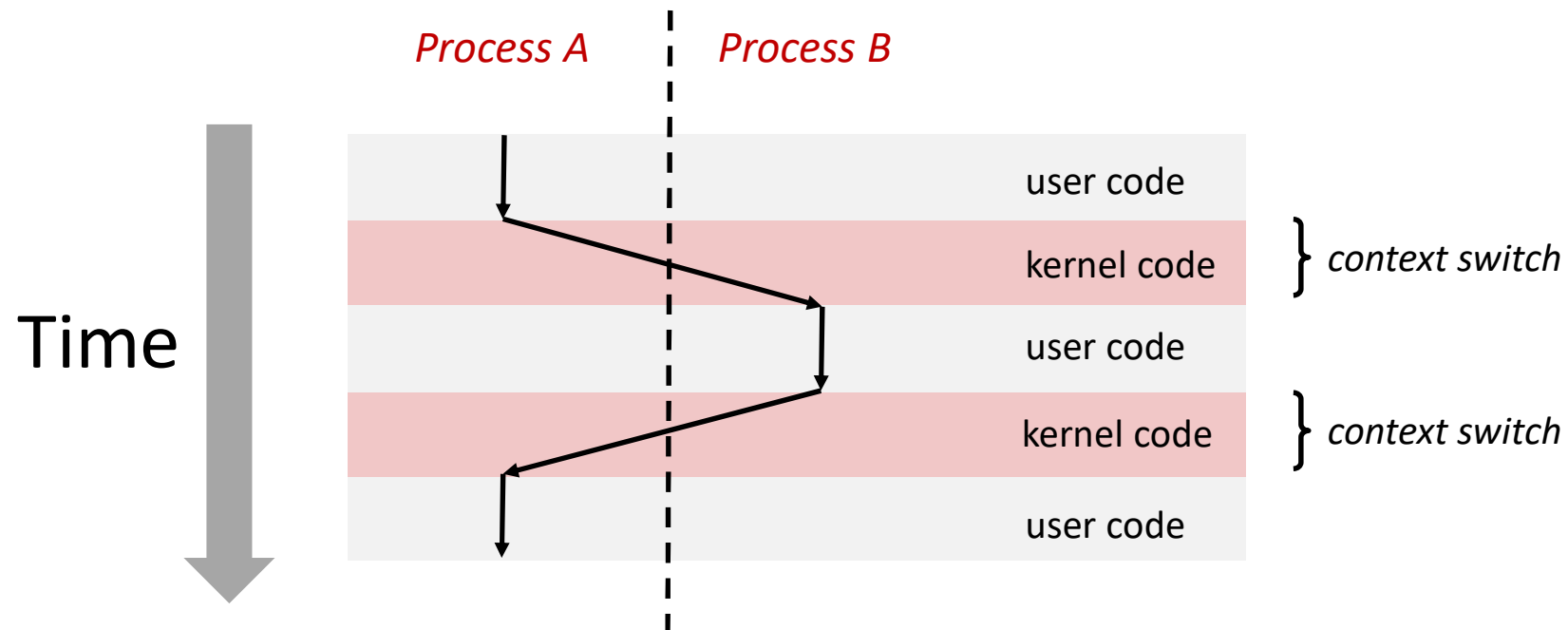
# User View of Concurrent Processes

- Control flows for concurrent processes are physically disjoint in time
- However, we can think of concurrent processes as running in parallel with each other



# Context Switching

- Processes are managed by a shared chunk of OS code called the *kernel*
  - Important: the kernel is not a separate process, but rather runs as part of some user process
- Control flow passes from one process to another via a *context switch*



# fork: Creating New Processes

## CreateProcess: Windows

### ■ `int fork(void)`

- creates a new process (child process) that is identical to the calling process (parent process)
- returns 0 to the child process
- returns child's **pid** (process id) to the parent process

```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```

- Fork is interesting (and often confusing) because it is called *once* but returns *twice*
- *CreateProcess* creates a new, empty address space and loads the specified program into it from disk.

# Understanding fork

## *Process n*

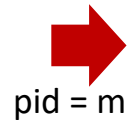


```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```

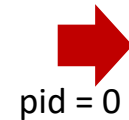
## *Child Process m*



```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```



```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```



```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```



```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```



```
pid_t pid = fork();
if (pid == 0) {
    printf("hello from child\n");
} else {
    printf("hello from parent\n");
}
```

hello from parent

*Which one is first?*

hello from child

# Fork Example #1

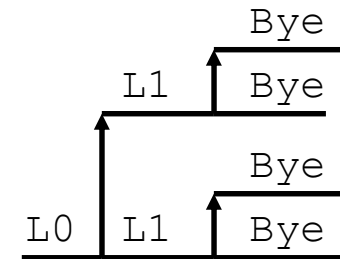
- **Parent and child both run same code**
  - Distinguish parent from child by return value from `fork`
- **Start with same state, but each has private copy**
  - Including shared output file descriptor
  - Relative ordering of their print statements undefined

```
void fork1()
{
    int x = 1;
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child has x = %d\n", ++x);
    } else {
        printf("Parent has x = %d\n", --x);
    }
    printf("Bye from process %d with x = %d\n", getpid(), x);
}
```

## Fork Example #2

- **Two consecutive forks**

```
void fork2()  
{  
    printf("L0\n");  
    fork();  
    printf("L1\n");  
    fork();  
    printf("Bye\n");  
}
```

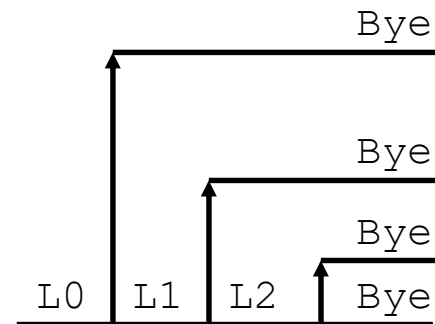


- **Windows: CreateProcess()** is complex, requiring ten different parameters to specify the executable path, command-line arguments, security attributes, and process flags.
- Handles (like open files) the child should inherit can be chosen, but the new process is a fresh operation compared to the total copy in Linux.
- Windows programmers use **threads** instead of processes for parallel programming.

# Fork Example #3

## ■ Nested forks in parent

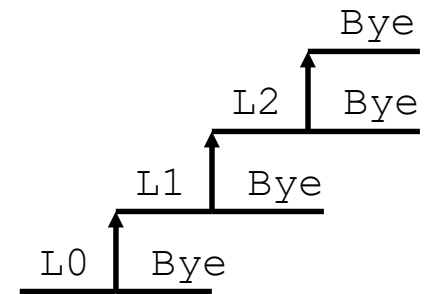
```
void fork4()  
{  
    printf("L0\n");  
    if (fork() != 0) {  
        printf("L1\n");  
        if (fork() != 0) {  
            printf("L2\n");  
            fork();  
        }  
    }  
    printf("Bye\n");  
}
```



# Fork Example #4

## ■ Nested forks in children

```
void fork5()  
{  
    printf("L0\n");  
    if (fork() == 0) {  
        printf("L1\n");  
        if (fork() == 0) {  
            printf("L2\n");  
            fork();  
        }  
    }  
    printf("Bye\n");  
}
```





# exit: Ending a process

- **void exit(int status)**

- exits a process
  - Normally return with status 0
- **atexit()** registers functions to be executed upon exit

```
void cleanup(void) {  
    printf("cleaning up\n");  
}  
  
void fork6() {  
    atexit(cleanup);  
    fork();  
    exit(0);  
}
```

# Zombies

## ■ Idea

- When process terminates, still consumes system resources
  - Various tables maintained by OS
- Called a “zombie”
  - Living corpse, half alive and half dead

## ■ Reaping

- Performed by parent on terminated child (using `wait` or `waitpid`)
- Parent is given exit status information
- Kernel discards process

## ■ What if parent doesn't reap?

- If any parent terminates without reaping a child, then child will be reaped by `init` process (`pid == 1`)
- So, only need explicit reaping in long-running processes
  - e.g., shells and servers

# Zombie Example

```
linux> ./forks 7 &
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9        00:00:00 tcsh
 6639 ttyp9        00:00:03 forks
 6640 ttyp9        00:00:00 forks <defunct>
 6641 ttyp9        00:00:00 ps
linux> kill 6639
[1] Terminated
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9        00:00:00 tcsh
 6642 ttyp9        00:00:00 ps
```

```
void fork7()
{
    if (fork() == 0) {
        /* Child */
        printf("Terminating Child, PID = %d\n",
            getpid());
        exit(0);
    } else {
        printf("Running Parent, PID = %d\n",
            getpid());
        while (1)
            ; /* Infinite loop */
    }
}
```

- **ps** shows child process as “defunct”
- Killing parent allows child to be reaped by **init**

# Nonterminating Child Example

```
void fork8()
{
    if (fork() == 0) {
        /* Child */
        printf("Running Child, PID = %d\n",
               getpid());
        while (1)
            ; /* Infinite loop */
    } else {
        printf("Terminating Parent, PID = %d\n",
               getpid());
        exit(0);
    }
}
```

```
linux> ./forks 8
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
  PID TTY          TIME CMD
 6585 tttyp9      00:00:00 tcsh
 6676 tttyp9      00:00:06 forks
 6677 tttyp9      00:00:00 ps
linux> kill 6676
linux> ps
  PID TTY          TIME CMD
 6585 tttyp9      00:00:00 tcsh
 6678 tttyp9      00:00:00 ps
```

- Child process still active even though parent has terminated
- Must kill explicitly, or else will keep running indefinitely

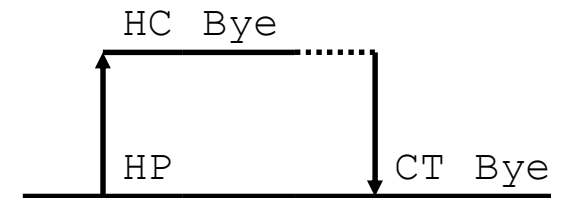
# **wait: Synchronizing with Children**

- Parent reaps child by calling the **wait** function
- **int wait(int \*child\_status)**
  - suspends current process until one of its children terminates
  - return value is the **pid** of the child process that terminated
  - if **child\_status != NULL**, then the object it points to will be set to a status indicating why the child process terminated

# wait: Synchronizing with Children

```
void fork9() {
    int child_status;

    if (fork() == 0) {
        printf("HC: hello from child\n");
    }
    else {
        printf("HP: hello from parent\n");
        wait(&child_status);
        printf("CT: child has terminated\n");
    }
    printf("Bye\n");
    exit();
}
```



# wait() Example

- If multiple children completed, will take in arbitrary order
- Can use macros WIFEXITED and WEXITSTATUS to get information about exit status

```
void fork10()
{
    pid_t pid[N];
    int i;
    int child_status;
    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100+i); /* Child */
    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

# waitpid() : Waiting for a Specific Process

## ■ waitpid(pid, &status, options)

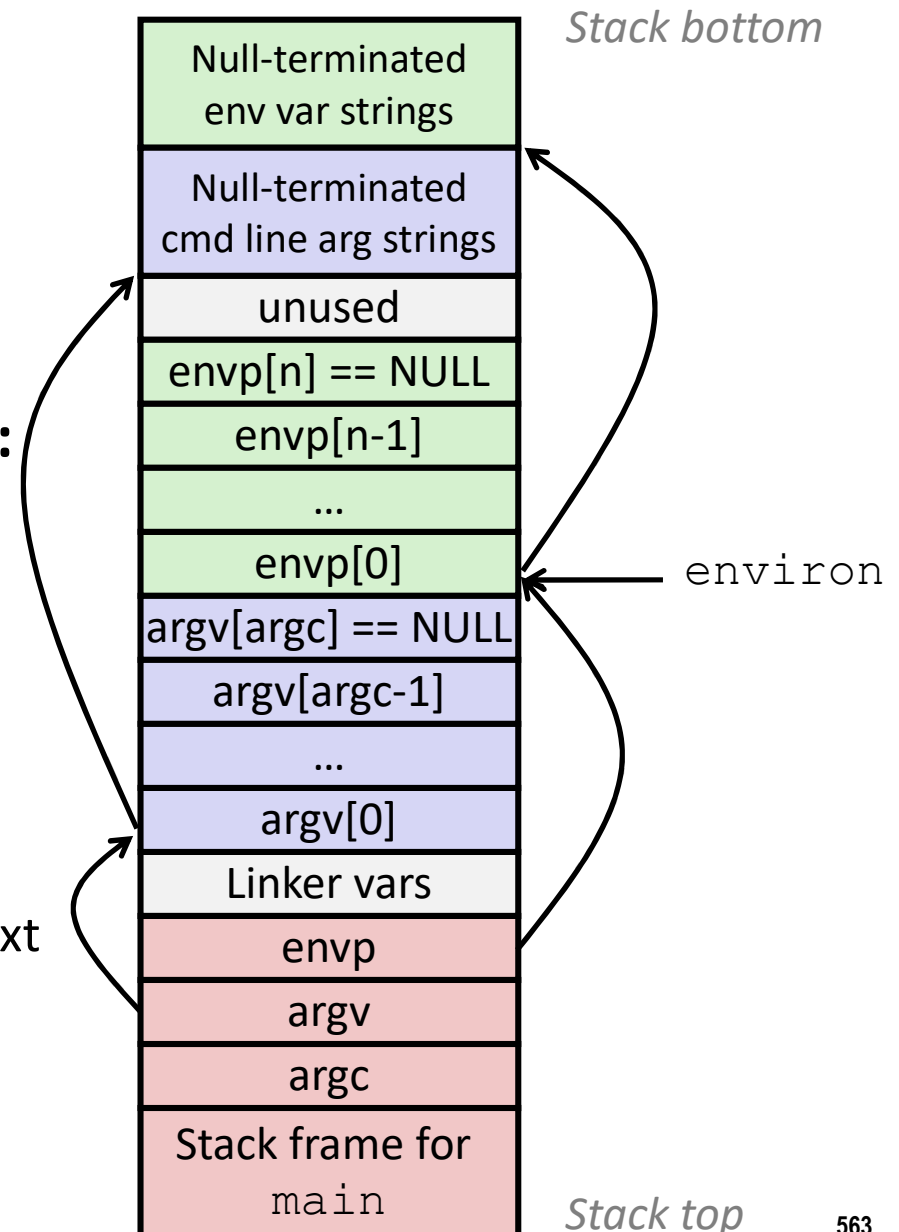
- suspends current process until specific process terminates
- various options

```
void fork11()
{
    pid_t pid[N];
    int i;
    int child_status;
    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100+i); /* Child */
    for (i = N-1; i >= 0; i--) {
        pid_t wpid = waitpid(pid[i], &child_status, 0);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```



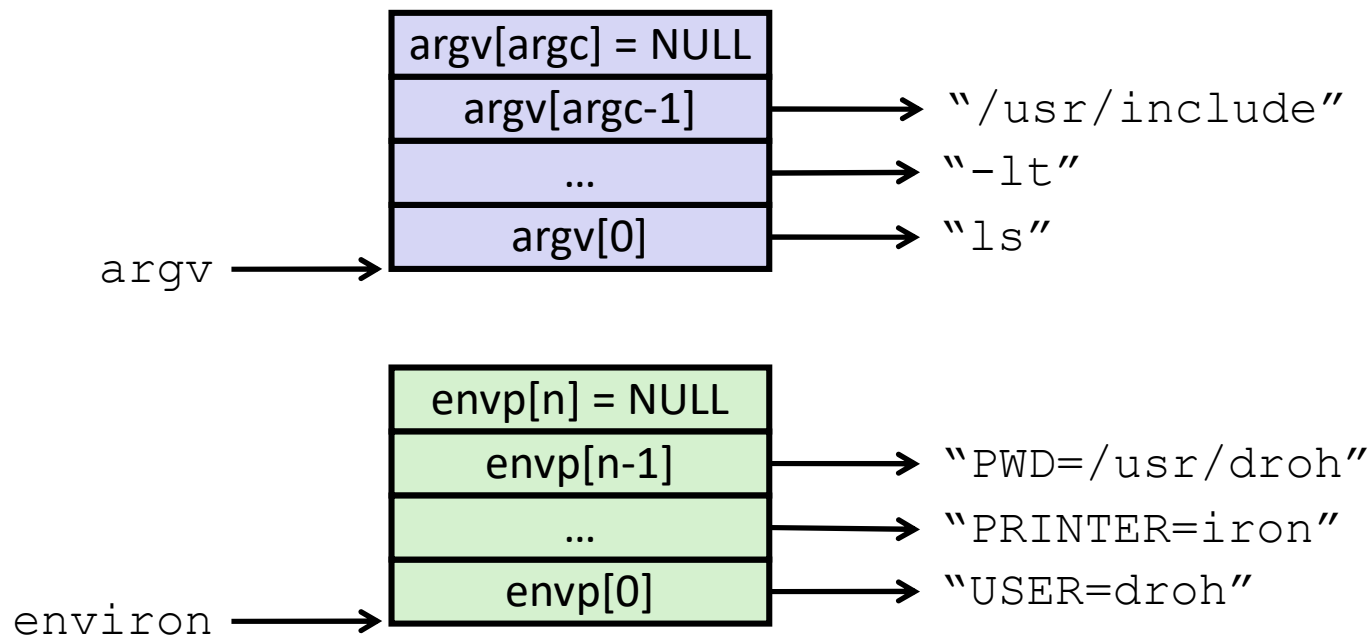
# execve : Loading and Running Programs

- `int execve(`  
    `char *filename,`  
    `char *argv[],`  
    `char *envp[]`  
)
- **Loads and runs in current process:**
  - Executable `filename`
  - With argument list `argv`
  - And environment variable list `envp`
- **Does not return (unless error)**
- **Overwrites code, data, and stack**
  - keeps pid, open files and signal context
- **Environment variables:**
  - “name=value” strings
  - `getenv` and `putenv`



# execve Example

```
if ((pid = Fork()) == 0) { /* Child runs user job */
    if (execve(argv[0], argv, environ) < 0) {
        printf("%s: Command not found.\n", argv[0]);
        exit(0);
    }
}
```



# The World of Multitasking

- **System runs many processes concurrently**
- **Process: executing program**
  - State includes memory image + register values + program counter
- **Regularly switches from one process to another**
  - Suspend process when it needs I/O resource or timer event occurs
  - Resume process when I/O available or given scheduling priority
- **Appears to user(s) as if all processes executing simultaneously**
  - Even though most systems can only execute one process at a time
  - Except possibly with lower performance than if running alone

# Programmer's Model of Multitasking (Linux)

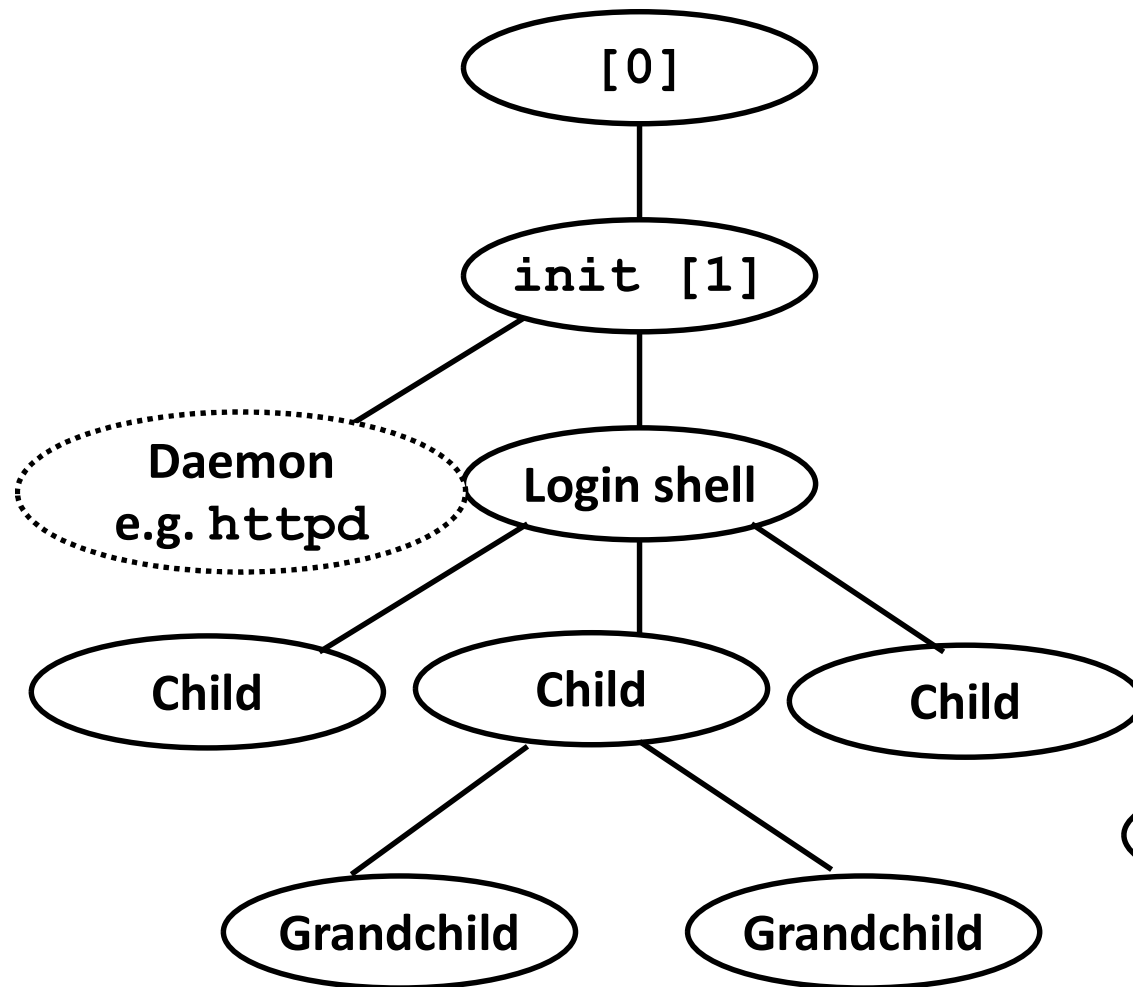
## ■ Basic functions

- **fork** spawns new process
  - Called once, returns twice
- **exit** terminates own process
  - Called once, never returns
  - Puts it into “zombie” status
- **wait** and **waitpid** wait for and reap terminated children
- **execve** runs new program in existing process
  - Called once, (normally) never returns

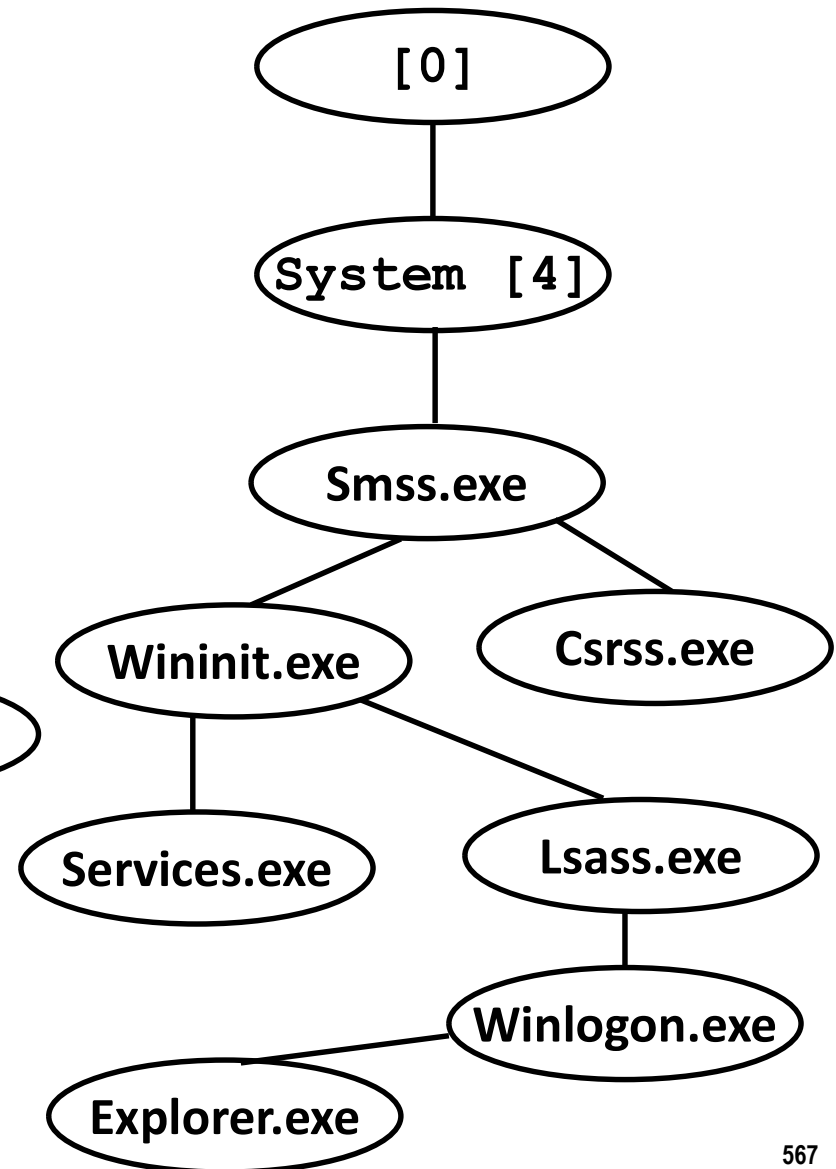
## ■ Programming challenge

- Understanding the nonstandard semantics of the functions
- Avoiding improper use of system resources
  - E.g. “Fork bombs” can disable a system

# Unix Process Hierarchy



# Windows



# Multitasking (Windows)

- **A thread is the smallest unit of execution that can be scheduled by the operating system.**
  - **Process:** A container for resources (address space, file handles, security tokens). A process "owns" things.
  - **Thread:** The "worker" inside the container. Every process has at least one thread (the main thread).
- **Shared Resources:** All threads within a single process share the same memory (heap) and files, but they have their own execution state.
- **The Thread Object:** In the Windows kernel, each thread is represented by an ETHREAD (Executive Thread) block and a KTHREAD (Kernel Thread) block.

# Multitasking (Windows)

- **The Thread Object:** In the Windows kernel, each thread is represented by an **ETHREAD** (Executive Thread) block and a **KTHREAD** (Kernel Thread) block.
- **What a Thread Owns:**
  - **Stack:** Two separate stacks—one for User Mode and one for Kernel Mode (to prevent security leaks).
  - **Thread Local Storage (TLS):** Private memory unique to that thread.
  - **Context:** The current state of CPU registers (EIP/RIP, Stack Pointer, etc.).
  - **TEB (Thread Environment Block):** A data structure in user mode used by the thread to manage its own state.

# Multitasking (Windows)

- **Multitasking:** The Windows scheduler doesn't wait for a thread to stop. It forcibly interrupts threads when their time is up.
- **Priority Levels (0–31):**
  - **0:** Idle (System only).
  - **1–15:** Dynamic Priorities (Standard apps).
  - **16–31:** Real-Time Priorities (Critical OS tasks, audio processing).
- **Priority Boosting:** Windows automatically "boosts" the priority of a thread if it's in the foreground or if it has just finished waiting for I/O, making the system feel more responsive.



# Multitasking (Windows)

- **The Race Condition:** Since threads share the same memory, two threads trying to update the same variable at the same time can cause data corruption.
- **Windows Synchronization Primitives:**
  - **Mutex (Mutual Exclusion):** Only one thread can "own" a resource at a time.
  - **Semaphores:** Allows a specific number of threads (e.g., 5) to access a resource.
  - **Critical Sections:** A high-speed, user-mode locking mechanism for threads within the *same* process.
  - **Events:** A way for one thread to "signal" another that a task is finished.

# Shell Programs

- A *shell* is an application program that runs programs on behalf of the user.
  - **sh** Original Unix shell (Stephen Bourne, AT&T Bell Labs, 1977)
  - **csh** BSD Unix C shell (**tcsh**: enhanced **csh** at CMU and elsewhere)
  - **bash** “Bourne-Again” Shell

```
int main() {
    char cmdline[MAXLINE];

    while (1) {
        /* read */
        printf("> ");
        Fgets(cmdline, MAXLINE, stdin);
        if (feof(stdin))
            exit(0);

        /* evaluate */
        eval(cmdline);
    }
}
```

*Execution is a sequence of  
read/evaluate steps*

Windows  
cmd.exe

# What Is a “Background Job”?

- **Users generally run one command at a time**
  - Type command, read output, type another command
- **Some programs run “for a long time”**
  - Example: “delete this file in two hours”

```
unix> sleep 7200; rm /tmp/junk # shell stuck for 2 hours
```

- **A “background” job is a process we don't want to wait for**

```
unix> (sleep 7200 ; rm /tmp/junk) &  
[1] 907  
unix> # ready for next command
```

- **A background process is an executable that runs without direct user interaction or a graphical user interface (GUI).** They handle system-level tasks (disk indexing, network monitoring, security). Unlike foreground apps, they do not have a window and often start automatically during the boot sequence.

# Background Processes

- **Service Control Manager (SCM):** A special system process (**services.exe**) that acts as the "boss" for background tasks.
  - Can run even when no user is logged in.
  - Often run under specialized accounts like LocalSystem, NetworkService, or LocalService to minimize security risks.
- In older OS Windows designs, every service was its own .exe. This wasted memory because of redundant overhead.
- **Solution:** svchost.exe is a generic host process name for services that run from dynamic-link libraries (DLLs). Windows groups related services into a single "host" process to save RAM.
- **Time Slicing:** Background processes share the same CPU as foreground apps. The Windows Kernel uses Preemptive Multitasking to give each process a "quantum" (time slice).

# User Mode vs. Kernel Mode

- **User Mode (Ring 3):** Most background processes run here. If they crash, they don't take down the whole system. They have restricted access to hardware.
- **Kernel Mode (Ring 0):** Reserved for critical drivers and the core OS. If a background process (like a low-level driver) running here fails, it results in a system crash.
- **System Calls:** Background processes must use *System Calls* to request resources (like writing to a log file on the SSD) from the Kernel.